

Journal de Bord — Stage PFE

Stagiaire	Mourad Slim JNAYEH
Entreprise	Keyrus
Période	Janvier 2026 — Juin 2026
Projet	Qatra — Plateforme de Gestion des Dons de Sang

Comment je m'organise pour mon stage

Avant même de commencer le développement, j'ai décidé de structurer mon stage en 8 sprints suivant la méthodologie Scrum. Chaque matin, je faisais un point rapide sur ce que j'avais à faire dans la journée avec l'encadrant, et chaque vendredi, je faisais un bilan de ce que j'avais accompli par rapport à ce que j'avais prévu. Cette organisation m'a permis de rester concentré et de ne pas me disperser, surtout quand le projet est devenu complexe avec les microservices.

Pour gérer les délais, je répartissais les tâches techniques (backend, frontend, tests) sur les jours de la semaine en alternant pour ne pas m'épuiser sur un seul sujet. Quand je réalisais qu'un sprint prenait plus de temps que prévu, je décalais les tâches non critiques vers le sprint suivant plutôt que de sacrifier la qualité.

Sprint	From	To	Duration	What to Do
Sprint 0	January 1	January 15	15 days	Conception & Architecture
Sprint 1	January 16	February 5	21 days	Authentication & Donation Centers
Sprint 2	February 6	February 26	21 days	Donation Management
Sprint 3	February 27	March 19	21 days	Appointment System
Sprint 4	March 20	April 9	21 days	Staff Operations & Admin Features
Sprint 5	April 10	May 9	30 days	Emergency Response & Super Admin
Sprint 6	May 10	June 8	30 days	Matching Engine & Notification Engine
Sprint 7	June 9	June 29	21 days	Analytics & Finalization

Introduction

Ce journal de bord retrace le déroulement de mon stage de fin d'études au sein de Keyrus, d'une durée de six mois allant de janvier à juin 2026. Le projet réalisé est Qatra, une plateforme distribuée de gestion des dons de sang connectant donneurs, centres de donation, personnel médical et administrateurs. Le développement suit une approche Scrum en 8 sprints. Ce document consigne les activités réalisées par sprint, les compétences acquises, les difficultés rencontrées et les réflexions personnelles.

Sprint 0 — Conception & Architecture

1 Janvier — 15 Janvier 2026 (15 jours)

Qu'ai-je appris ?

Ce premier sprint m'a appris que la conception est souvent plus difficile que le développement lui-même. J'ai découvert l'architecture hexagonale (ports et adaptateurs) et j'ai compris pourquoi on l'utilise : elle permet de séparer la logique métier de l'infrastructure, ce qui rend le code plus testable et maintenable. J'ai aussi appris à concevoir un diagramme de déploiement Docker Compose et à planifier des pipelines CI/CD, des choses que je

n'avais jamais faites en projet réel.

Quelles missions ai-je accomplies ?

J'ai conçu l'architecture complète du système Qatra, qui repose sur deux microservices communiquant via Apache Kafka :

- donation-service : le service principal gérant les utilisateurs, les centres, les rendez-vous, les urgences, les analytics
- notification-service : un microservice dédié aux notifications multi-canal (email, temps réel)

Pour le donation-service, j'ai défini 10 modules hexagonaux : user, donor, center, appointment, emergency, analytics, system, _shared, _config, report.

J'ai aussi modélisé la base de données avec 22 tables PostgreSQL et conçu les endpoints REST pour 65 cas d'utilisation au total. J'ai créé des wireframes pour les pages principales (Landing Page, Dashboard Donneur, Réservation RDV, Urgence, Admin Dashboard) et un diagramme de déploiement Docker Compose avec 12 services.

Quelles compétences ai-je utilisées ?

- Conception d'architecture logicielle distribuée
- Modélisation UML (diagramme de classes)
- Compréhension des patterns CQRS et Hexagonal
- Conception de schémas de base de données relationnelle
- Conception d'API RESTful

À quelles difficultés ai-je dû faire face ?

La conception du moteur de correspondance géospatiale pour les urgences m'a posé problème. Je devais réfléchir à un algorithme de recherche radiale avec rayon progressif, et je ne savais pas par où commencer. De plus, l'isolation des modules via Spring Modulith imposait des contraintes strictes sur les dépendances inter-modules, ce qui rendait la conception plus complexe.

Ce que j'ai fait pour surmonter ces difficultés : J'ai beaucoup lu sur les algorithmes de recherche géospatiale et j'ai opté pour une approche par paliers : chercher d'abord dans un rayon de 10km, puis élargir progressivement si pas assez de résultats. Pour Spring Modulith, j'ai étudié la documentation officielle et j'ai dessiné un diagramme des dépendances entre modules avant de coder, ce qui m'a évité de créer des couplages interdits.

Sprint 1 — Authentification & Centres de Donation

16 Janvier — 5 Février 2026 (21 jours)

Qu'ai-je appris ?

J'ai beaucoup appris sur la sécurité applicative. Avant ce sprint, je savais ce qu'était un JWT en théorie, mais je n'en avais jamais implémenté un système complet avec access tokens et refresh tokens. J'ai compris l'importance de hasher les tokens en base, de gérer les sessions, et de révoquer les tokens lors du logout. J'ai aussi découvert comment Spring Security s'intègre avec l'architecture hexagonale : il faut séparer les adaptateurs de sécurité des ports métier.

Quelles missions ai-je accomplies ?

Backend — Authentification & Utilisateurs :

- Module user : CRUD utilisateur avec Spring Security et JWT stateless
- Authentification complète : login, inscription, vérification email, réinitialisation mot de passe, logout
- Gestion des sessions : création de Session avec access/refresh tokens hashés, validation, refresh, révocation
- Gestion des 4 rôles : DONOR, CENTER_STAFF, CENTER_ADMIN, SUPER_ADMIN
- Chiffrement BCrypt pour les mots de passe

Backend — Centres de Donation :

- Module center : CRUD DonationCenter avec 4 statuts (PENDING_APPROVAL, ACTIVE, SUSPENDED, CLOSED)
- Soumission, mise à jour, configuration de capacité, gestion des horaires d'ouverture et fermetures

Frontend :

- Pages Angular : Login, Inscription, Vérification email, Mot de passe oublié
- Intercepteurs HTTP pour le JWT (token storage, refresh automatique)
- Signal stores NgRx/signals pour la gestion d'état
- Page de listing des centres avec affichage cartographique (MapLibre GL)

Quelles compétences ai-je utilisées ?

- Spring Security avec JWT stateless
- Gestion des sessions et des refresh tokens
- Architecture hexagonale appliquée à la sécurité
- Angular (standalone components, intercepteurs HTTP, Zod validation)
- Intégration de cartes (MapLibre GL)

À quelles difficultés ai-je dû faire face ?

La gestion du refresh token nécessitait une synchronisation précise entre la durée de vie du token et la révocation de session. Si le timing n'était pas bon, l'utilisateur était déconnecté trop tôt ou trop tard. La configuration de Spring Security avec l'architecture hexagonale imposait de séparer les adaptateurs de sécurité des ports métier, ce qui n'était pas intuitif.

Ce que j'ai fait pour surmonter ces difficultés : J'ai dessiné un diagramme de séquence du flux de login !' refresh !' logout pour bien comprendre les interactions. Pour Spring Security, j'ai créé un adaptateur de sécurité dédié dans le port d'infrastructure, ce qui respectait l'architecture hexagonale tout en gardant la logique de sécurité propre.

Sprint 2 — Gestion des Donations

6 Février — 26 Février 2026 (21 jours)

Qu'ai-je appris ?

J'ai appris à implémenter un workflow métier complexe avec beaucoup de règles. Le scoring de fiabilité m'a particulièrement intéressé : il fallait équilibrer plusieurs facteurs (historique de donation, rendez-vous manqués, régularité) dans un seul score de 0 à 100. J'ai aussi compris l'importance de la conformité RGPD : quand un utilisateur demande la suppression de ses données, on ne les supprime pas immédiatement, on attend 30 jours puis on les anonymise. Cette contrainte légale a ajouté une couche de complexité au développement.

Quelles missions ai-je accomplies ?

Backend :

- Module donateur : profil donateur, questionnaire de santé, éligibilité, scoring de fiabilité
- Onboarding complet : profil santé (maladies chroniques, chirurgies, voyages, tatouages), groupe sanguin, préférences de notification, localisation, statut de disponibilité
- Demande de suppression de compte RGPD avec workflow en 3 étapes
- Vérification automatique de l'éligibilité (cooldown de 56 jours entre donations)
- Scoring de fiabilité : ajustement du score sur complétion, no-show, déclinaisons d'urgences
- Dashboard donateur avec score de fiabilité, prochaine date d'éligibilité, historique
- Génération de certificats de donation en PDF (OpenPDF)

Frontend :

- Tableau de bord personnel du donateur
- Formulaire de questionnaire de santé
- Page des certificats de donation avec téléchargement PDF
- Page d'impact avec visualisation des donations cumulées

Quelles compétences ai-je utilisées ?

- Modélisation de workflow métier avec machine à états
- Gestion de la conformité RGPD (anonymisation, suppression)
- Génération de PDF (OpenPDF)
- Scoring algorithmique multi-facteurs

À quelles difficultés ai-je dû faire face ?

L'algorithme de scoring de fiabilité devait équilibrer plusieurs facteurs sans qu'un seul domine les autres. Le workflow RGPD avec suppression demandée !' attente 30 jours !' anonymisation nécessitait une gestion rigoureuse des statuts.

Ce que j'ai fait pour surmonter ces difficultés : Pour le scoring, j'ai commencé par un prototype simple (poids égaux) puis j'ai ajusté les poids en testant différents scénarios. Pour le RGPD, j'ai dessiné un diagramme d'états du compte utilisateur avec tous les transitions possibles, ce qui m'a permis de ne rien oublier.

Sprint 3 — Système de Rendez-vous

27 Février — 19 Mars 2026 (21 jours)

Qu'ai-je appris ?

J'ai appris à gérer la concurrence sur des ressources partagées. Quand deux donneurs réservent le même créneau en même temps, il fallait un mécanisme de verrouillage pour éviter le dépassement de capacité. J'ai découvert le verrouillage optimiste avec JPA (annotation @Version) et j'ai compris comment ça fonctionne en pratique. J'ai aussi appris à configurer une stack d'observabilité complète (Prometheus, Grafana, Loki, Jaeger) pour monitorer l'application.

Quelles missions ai-je accomplies ?

Backend :

- Module appointment : workflow complet de réservation
- Réservation : sélection d'un centre, choix d'un créneau, création d'un Appointment
- Créneaux horaires (Slot) : génération automatique toutes les 21 jours, gestion de la capacité
- Report et annulation avec libération de capacité
- Check-in par QR code ou sélection manuelle
- Rappels automatiques quotidiens (8h pour les rendez-vous, 9h pour l'éligibilité)
- Expiration automatique des urgences non résolues

Frontend :

- Page de réservation : sélection centre !' calendrier interactif !' créneau !' confirmation
- QR code de check-in pour chaque rendez-vous

Monitoring :

- Stack complète : Prometheus + Grafana + Loki + Promtail + Jaeger
- Métriques Micrometer et tracing OpenTelemetry

Quelles compétences ai-je utilisées ?

- Gestion de concurrence (verrouillage optimiste JPA)
- Génération automatique de créneaux
- Stack d'observabilité (Prometheus, Grafana, Loki, Jaeger)
- QR code (génération et scan)

À quelles difficultés ai-je dû faire face ?

La gestion de la concurrence lors de réservations simultanées sur le même créneau nécessitait des mécanismes de verrouillage. La synchronisation entre la génération automatique de créneaux et les fermetures exceptionnelles demandait une planification minutieuse.

Ce que j'ai fait pour surmonter ces difficultés : J'ai implémenté un verrouillage optimiste avec @Version sur l'entité Slot et j'ai géré les OptimisticLockException en retentant la réservation. Pour la synchronisation des créneaux, j'ai créé un scheduler qui vérifie les fermetures avant de générer les créneaux, et j'ai ajouté des tests unitaires pour couvrir tous les cas limites.

Sprint 4 — Opérations Staff & Fonctionnalités Admin

20 Mars — 9 Avril 2026 (21 jours)

Qu'ai-je appris ?

J'ai appris à concevoir des interfaces pour différents profils utilisateurs. Le personnel médical a besoin d'une interface rapide et efficace (agenda quotidien, file d'attente), tandis que l'administrateur a besoin de vues agrégées (reporting, tendances). J'ai aussi compris l'importance des permissions granulaires : un staff ne doit pas pouvoir faire la même chose qu'un admin, même s'ils travaillent dans le même centre.

Quelles missions ai-je accomplies ?

Backend — Staff Operations :

- Consultation de l'agenda quotidien et de la file d'attente
- Check-in du donneur avec consultation du profil santé et éligibilité
- Dépistage de santé : création d'un HealthScreening (poids, pression, hémoglobine, température)
- Finalisation du rendez-vous avec enregistrement du volume collecté
- Signalement d'absence (no-show) avec déduction automatique du score de fiabilité

Backend — Admin Features :

- Gestion du centre : mise à jour, capacité, fermetures
- Gestion du personnel : ajout/suppression de CENTER_STAFF
- Reporting : tendances, heures de pointe, inventaire par groupe sanguin

Frontend :

- Interface Staff : agenda quotidien, file d'attente, détails du rendez-vous
- Workflow de dépistage de santé
- Tableau de bord Admin : gestion du centre, capacité, personnel

Quelles compétences ai-je utilisées ?

- Machine à états pour le workflow de dépistage
- Permissions granulaires par rôle
- Conception d'interfaces multi-profils

À quelles difficultés ai-je dû faire face ?

Le workflow de dépistage de santé avec ses différentes issues (éligible/inéligible, temporaire/permanent) nécessitait une machine à états bien définie. La gestion des rôles avec des permissions granulaires imposait une vérification fine à chaque endpoint.

Ce que j'ai fait pour surmonter ces difficultés : J'ai dessiné un diagramme d'états complet du dépistage avant de coder, ce qui m'a évité de devoir refactoriser ensuite. Pour les permissions, j'ai créé des annotations Spring Security personnalisées (@RequiresRole) et j'ai écrit des tests pour chaque combinaison rôle/endpoint.

Sprint 5 — Réponse d'Urgence & Super Admin

10 Avril — 9 Mai 2026 (30 jours)

Qu'ai-je appris ?

J'ai appris à implémenter un système d'urgence en temps réel, ce qui était le défi technique le plus stimulant du projet. La coordination entre le scheduler d'escalade, le moteur de correspondance géospatiale et le système de notifications m'a appris à penser en termes d'événements et de flux asynchrones. J'ai aussi compris l'importance du journal d'audit : chaque action critique doit être traçable.

Quelles missions ai-je accomplies ?

Backend — Emergency Response :

- Cycle de vie complet des urgences : création, suivi, escalade, résolution
- Création avec groupe sanguin, unités nécessaires, niveau d'urgence (CRITICAL à LOW), deadline
- Notification aux donneurs éligibles avec détails (centre, distance, groupe sanguin)
- Réponse du donneur : acceptation ou déclinaison avec raison optionnelle

- Allocation de créneau d'urgence et résolution automatique
- Expiration automatique des urgences dépassant le deadline
- Journal d'audit pour chaque changement d'état

Backend — Super Admin :

- Gestion des utilisateurs : recherche, filtrage, activation/suspension/suppression
- Gestion des centres : approbation/rejet des demandes d'enregistrement
- Traitement RGPD : workflow complet avec suppression demandée !' attente 30 jours !' anonymisation
- Tableau de bord système : urgences, taux de réponse, stats donneurs

Frontend :

- Module Urgence : création, détail, liste des urgences actives
- Réponse du donneur : vue de l'alerte, acceptation/déclinaison
- Super Admin UI : gestion des utilisateurs, approbation des centres, rôles, audit logs

Quelles compétences ai-je utilisées ?

- Système d'urgence en temps réel
- Escalade automatique avec rayon progressif
- Journal d'audit (traçabilité)
- Machine à états pour le workflow RGPD

À quelles difficultés ai-je dû faire face ?

L'implémentation de l'escalade automatique avec rayon progressif (10km par niveau, max 200km) nécessitait une coordination entre le scheduler et le moteur de correspondance. La gestion du workflow RGPD avec ses 30 jours d'attente et la possibilité d'annulation par le donneur exigeait une machine à états robuste.

Ce que j'ai fait pour surmonter ces difficultés : Pour l'escalade, j'ai créé un scheduler Spring qui s'exécute toutes les minutes, vérifie les urgences non résolues, et déclenche l'élargissement du rayon. J'ai testé chaque niveau d'escalade individuellement avant de les enchaîner. Pour le RGPD, j'ai dessiné un diagramme d'états complet incluant toutes les transitions (demande, attente, annulation, anonymisation) et j'ai écrit des tests pour chaque chemin.

Sprint 6 — Moteur de Correspondance & Moteur de Notification

10 Mai — 8 Juin 2026 (30 jours)

Qu'ai-je appris ?

J'ai appris à concevoir un moteur de correspondance géospatiale performant. L'algorithme de scoring qui combine distance, fiabilité et historique de donation m'a appris à pondérer plusieurs critères objectifs. J'ai aussi découvert Apache Kafka en profondeur : la gestion des topics, le partitionnement, la propagation des headers de tracing entre microservices. Le microservice de notification m'a appris à concevoir un système multi-canal avec retry et backoff exponentiel.

Quelles missions ai-je accomplies ?

Backend — Matching Engine :

- Scan et filtrage des donneurs éligibles (groupe sanguin, éligibilité, localisation, profil actif)
- Scoring et classement : fiabilité, distance, historique de donation
- Correspondance à paliers : escalade automatique si pas assez de donneurs
- Gestion de la déclinaison : compteur de déclinaisons consécutives, flag pour review à 3 déclinaisons

Backend — Notification Engine :

- Microservice séparé avec 6 listeners Kafka
- Dispatch multi-canal : IN_APP (WebSocket STOMP/SockJS) + EMAIL (SendGrid, Resend, Gmail SMTP)
- 6 types de notifications : EMERGENCY_ALERT, APPOINTMENT_REMINDER, ELIGIBILITY_REMINDER, PROFILE_COMPLETION, PASSWORD_RESET, GENERAL
- Respect des QuietHours et des préférences de fréquence
- Suivi de livraison et retry avec backoff exponentiel

Frontend :

- Composant notification bell avec compteur de non-lus
- Page du centre de notifications avec filtrage par type
- Intégration WebSocket pour les notifications en temps réel

Quelles compétences ai-je utilisées ?

- Moteur de correspondance géospatiale
- Apache Kafka (producteur, consommateur, topics, partitionnement)
- WebSocket (STOMP/SockJS)
- Microservice de notification multi-canal
- Retry avec backoff exponentiel

À quelles difficultés ai-je dû faire face ?

La synchronisation des schémas d'événements Kafka entre les deux microservices nécessitait une gestion stricte des versions de contrats. La propagation des headers de tracing à travers Kafka était technique. La reconnexion automatique WebSocket en cas de déconnexion du client demandait une logique de fallback robuste.

Ce que j'ai fait pour surmonter ces difficultés : Pour Kafka, j'ai défini des schémas Avro pour les événements et j'ai utilisé un schema registry pour versionner les contrats. Pour le tracing, j'ai propagé manuellement les headers OpenTelemetry dans les messages Kafka. Pour WebSocket, j'ai implémenté une logique de reconnexion avec backoff exponentiel côté client Angular.

Configuration Kubernetes (20 Mai — 08 Juin)

Durant ce sprint, j'ai aussi mis en place l'orchestration Kubernetes (K3s) pour le déploiement en production. J'ai créé 16 fichiers de manifests déployés via un script `deploy.sh` en 8 étapes, avec `cert-manager` pour les certificats TLS, des `StatefulSets` pour les bases de données, des `Deployments` pour les services, et des `IngressRoutes` Traefik pour le routage.

J'ai appris que l'ordre de déploiement était critique : postgres d'abord, puis kafka, redis, puis les services. La configuration du cluster Kafka en KRaft sur K3s nécessitait un `CLUSTER_ID` fixe et l'adressage via le DNS `StatefulSet`.

Sprint 7 — Analytics & Finalisation

9 Juin — 29 Juin 2026 (21 jours)

Qu'ai-je appris ?

J'ai appris que la fin de projet est souvent la phase la plus stressante : il faut finaliser les fonctionnalités, corriger les bugs, optimiser les performances, et tout déployer en production. J'ai découvert l'importance des tests end-to-end pour détecter les bugs d'intégration entre microservices qui n'apparaissaient jamais en tests unitaires. J'ai aussi appris à documenter un projet pour qu'il soit maintenable par d'autres développeurs.

Quelles missions ai-je accomplies ?

Backend — Analytics :

- Métriques en temps réel (`MetricsSnapshot`) pour les tableaux de bord
- Prédiction de la demande par région et groupe sanguin
- Journaux d'audit avec `oldValue/newValue` pour chaque action
- Intégration Spring Actuator avec Prometheus

Frontend :

- Tableaux de bord analytics avec Chart.js
- Dashboard Super Admin : métriques système, santé des services
- Dashboard Center Admin : analytics du centre, heures de pointe

Finalisation :

- Final QA : recettage fonctionnel complet
- Tests de performance et de sécurité finaux
- UAT et Beta Testing avec les utilisateurs finaux
- Release Notes complètes, matériel de formation, documentation API finale

- Déploiement final sur K3s via deploy.sh
- Configuration CI/CD avec GitHub Actions
- Démonstration finale du projet
- Rédaction du rapport de stage

Quelles compétences ai-je utilisées ?

- Métriques et analytics (Spring Actuator, Prometheus)
- Prédiction de la demande (algorithmes d'agrégation)
- Tests end-to-end
- Documentation technique
- Déploiement en production

À quelles difficultés ai-je dû faire face ?

La corrélation des métriques entre les deux microservices nécessitait une propagation correcte des identifiants de corrélation. L'optimisation du cache Redis avec des patterns d'accès variés demandait une analyse fine. Les bugs d'intégration entre services n'apparaissaient que lors des tests end-to-end.

Ce que j'ai fait pour surmonter ces difficultés : Pour la corrélation, j'ai propagé un correlationId dans tous les headers Kafka et HTTP. Pour Redis, j'ai analysé les patterns d'accès et j'ai configuré des TTL différents selon la fréquence d'utilisation. Pour les bugs d'intégration, j'ai créé un environnement de test reproduisant la production avec Docker Compose, ce qui m'a permis de détecter les problèmes avant le déploiement.

Bilan des Compétences Développées

Compétences Techniques

Domaine	Détails
Backend Java/Spring	Java 21, Spring Boot 4.0.6, Spring Modulith, Spring Security, JPA/Hibernate, architecture hexagonale, CQRS, virtual threads
Frontend Angular	Angular 20, standalone components, Angular Signals, NgRx/signals, PrimeNG, TailwindCSS, MapLibre GL, Chart.js
Bases de données	PostgreSQL 16 (23 tables), Redis 7 (cache), requêtes JPA optimisées
Architecture distribuée	Apache Kafka (KRaft, 9 topics), microservices, communication inter-services événementielle
DevOps	Docker, Docker Compose (12 services), Nginx, CI/CD GitHub Actions, K3s, Traefik, cert-manager
Observabilité	Prometheus, Grafana, Loki, Promtail, Jaeger, OpenTelemetry, Micrometer
Sécurité	JWT (access/refresh tokens), BCrypt, conformité RGPD
Temps réel	WebSocket (STOMP/SockJS), notifications push multi-canal

Compétences Transversales

Domaine	Détails
Gestion de projet	Méthodologie Scrum, sprints de 2-4 semaines, daily stand-ups, rétrospectives
Organisation	Planification des sprints, gestion des priorités et des délais
Résolution de problèmes	Debugging multi-services, optimisation de performance, architecture distribuée
Communication	Rédaction technique, documentation, présentation de soutenance

Réflexions Personnelles

Mon parcours durant le stage

Ce stage de six mois à Keyrus a été une expérience transformatrice. En tant que développeur full-stack sur le projet Qatra, j'ai couvert l'intégralité du cycle de développement logiciel : de la conception architecturale (Sprint 0) au déploiement en production (Sprint 7), en passant par le développement backend/frontend, les tests, et la documentation.

Le format Scrum avec 8 sprints m'a permis de livrer de la valeur de manière incrémentale. Au début, j'avais du mal à estimer la durée des tâches, mais avec l'expérience des premiers sprints, j'ai appris à être plus réaliste dans mes estimations. J'ai aussi appris l'importance de ne pas tout faire parfaitement : parfois, un « good enough » livré à temps vaut mieux qu'un « parfait » livré en retard.

Défis relevés

Le principal défi a été de maîtriser une stack technologique de pointe (Spring Boot 4, Angular 20, Kafka en KRaft) tout en livrant un projet fonctionnel de grande envergure. L'architecture distribuée avec deux microservices communiquant via Kafka, le moteur de correspondance géospatiale à paliers, et le système de notifications multi-canal temps réel ont été les défis techniques les plus stimulants.

J'ai aussi eu du mal avec la gestion du temps au début du stage. Je voulais tout faire parfaitement, ce qui me faisait perdre du temps. J'ai appris à prioriser : d'abord le fonctionnel, ensuite l'optimisation. Cette leçon m'a été très utile pour le reste du stage.

Ce que j'aurais aimé améliorer

Certaines fonctionnalités avancées n'ont pas pu être entièrement finalisées : l'anonymisation automatique RGPD à 30 jours (le scheduler n'est pas encore actif), la gestion des heures calmes et de la fréquence dans le moteur de notification, et les métriques avancées de prédiction de la demande. Le scoring de fiabilité pourrait aussi être enrichi avec des paramètres plus granulaires.

J'aurais aussi aimé passer plus de temps sur les tests end-to-end et les tests de performance. Les tests unitaires étaient bien couverts, mais les tests d'intégration entre microservices manquaient de profondeur.

Mon stage et l'idée que je me faisais du métier

Avant ce stage, je pensais que le métier de développeur full-stack consistait principalement à écrire du code. Je me rends compte que c'est beaucoup plus que ça : il y a la conception, la documentation, les tests, le déploiement, la collaboration avec l'équipe, la gestion des priorités. J'ai aussi découvert l'importance de la maintenance : écrire du code propre et bien documenté est essentiel pour que d'autres développeurs puissent reprendre le projet.

Le côté le plus surprenant a été la quantité de réflexion nécessaire avant de coder. Chaque sprint commençait par une phase de conception (diagrammes, choix d'architecture) qui prenait parfois plus de temps que le développement lui-même. J'ai compris que « mesurer deux fois, couper une fois » est une sagesse qui s'applique parfaitement au développement logiciel.

Je recommande vivement ce type de stage pour tout étudiant souhaitant se spécialiser dans le développement full-stack avec une approche DevOps. Le projet Qatra, avec sa complexité technique réelle et son impact social positif (gestion des dons de sang), a rendu cette expérience particulièrement motivante.

Statistiques du Projet Qatra

Métrique	Valeur
Cas d'utilisation	65 (Donor: 30, Staff: 11, Admin: 10, Super Admin: 13, System: 17)
Tables PostgreSQL (donation DB)	22
Tables PostgreSQL (notification DB)	1
Topics Kafka	9
Sprints	8 (Sprint 0 à Sprint 7)
Manifests Kubernetes (K3s)	16 fichiers

Pods Kubernetes	13
IngressRoutes	8
Modules Spring Modulith	10

Technologies Utilisées

Couche	Technologies
Backend	Java 21, Spring Boot 4.0.6, Spring Modulith, Spring Security, JPA/Hibernate, MapStruct, Lombok
Frontend	Angular 20, PrimeNG, TailwindCSS, NgRx/signals, MapLibre GL, Chart.js, Zod
Base de données	PostgreSQL 16, Redis 7
Messaging	Apache Kafka (KRaft mode, Confluent)
Sécurité	JWT, BCrypt, Spring Security
Temps réel	WebSocket
Email	SendGrid, Resend, Gmail SMTP
PDF	OpenPDF
Monitoring	Prometheus, Grafana, Loki, Promtail, Jaeger, OpenTelemetry
Conteneurisation	Docker, Docker Compose, Nginx
Orchestration	K3s, Traefik IngressRoute, cert-manager (Let's Encrypt), Helm
CI/CD	GitHub Actions
Tests	JUnit 5, Mockito, H2, JaCoCo, Jasmine, Karma

Journal de Bord v2 — Stage PFE — Mourad Slim JNAYEH — Keyrus 2026

